Gráfok és gráfbejárások

Alapfogalmak, szélességi és mélységi bejárás, Tarry, Trémaux és Dijkstra algoritmusa

Tartalom

1. Gráfelméleti alapfogalmak

2. Szélességi bejárás (BFS)

3. Mélységi bejárás (DFS)

4. Labirintus-bejárás: Tarry és Trémaux

5. Dijkstra algoritmusa

6. Összefoglalás

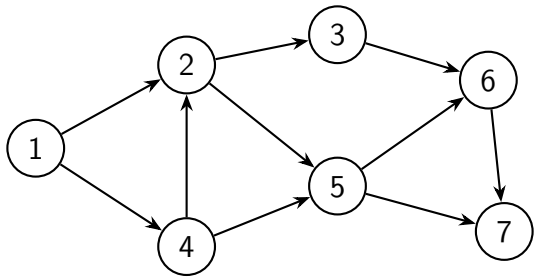
Mi a gráf?

Definíció

A gráf csúcsok véges halmaza, és egy ezen értelmezett élhalmaz: $G = (V, E)$.

- V: a csúcsok halmaza, pl. $\{1, 2, 3, 4, 5, 6, 7\}$
- E: az élek halmaza, pl. $\{(1, 2), (1, 4), (2, 3), \ldots\}$
- $|V|$: a csúcsok száma, $|E|$: az élek száma
- **irányított** gráf: az élek rendezett párok $(u \to v)$
- **irányítatlan** gráf: az élek rendezetlen párok
- **súlyozott** gráf: minden élhez tartozik egy $c(u, v)$ súly

A példagráfunk



Irányított gráf, $|V| = 7$, $|E| = 10$.

Élek: $1 \to 2$, $1 \to 4$, $2 \to 3$, $2 \to 5$, $3 \to 6$, $4 \to 2$, $4 \to 5$, $5 \to 6$, $5 \to 7$, $6 \to 7$.

További alapfogalmak

- **fokszám:** a csúcsra illeszkedő élek száma; irányított gráfnál **befok** és **kifok**
- **séta:** egymáshoz csatlakozó élek sorozata (csúcs és él ismétlődhet)
- **út:** olyan séta, amelyben nem ismétlődik csúcs
- **kör:** olyan séta, amely ugyanabba a csúcsba tér vissza
- **fa:** összefüggő, körmentes irányítatlan gráf
- **feszítőfa:** a gráf összes csúcsát tartalmazó fa részgráf

Összefüggőség

Irányítatlan gráf

Összefüggő, ha bármely két csúcsa összeköthető úttal.

Irányított gráf

Erősen összefüggő, ha bármely két csúcs esetén mindegyik elérhető a másikból.

- Ennek megfelelően beszélünk **összefüggő**, illetve **erősen összefüggő komponensekről**.
- **DAG:** irányított, körmentes gráf.

Gráfokon értelmezett alapműveletek

- `G.Csúcsok` – a gráf csúcsai
- `G.Élek` – a gráf élei
- `x.VezetÉl(y)` – vezet-e él x-ből y-ba?
- `x.Szomszédok` – azok a csúcsok, amelyekbe vezet él x-ből
- `x.Súly(y)` – súlyozott gráfnál az $x \to y$ él súlya
- `x.Tart` – a csúcshoz kapcsolt tartalom (adat)

Az algoritmusok szempontjából csak az számít, hogy a tárolási mód ezeket a műveleteket támogassa.

Szomszédsági lista

- Egy L tömb, melynek mérete a csúcsok száma.
- $L[i]$ egy láncolt lista: az i. csúcs szomszédai.
- Üres lista: nincs kivezető él.
- Súlyozott gráfnál a listaelem a súlyt is tárolja.

csúcs	szomszédok
1	2, 4
2	3, 5
3	6
4	2, 5
5	6, 7
6	7
7	–

Helyfoglalás: $|V|\cdot$ mutató $+$ $|E|\cdot$ (azonosító $+$ mutató).

Szomszédsági mátrix (csúcsmátrix)

- $CS[i, j] = 1$, ha vezet él i-ből j-be, különben 0.
- Irányítatlan gráfnál a mátrix szimmetrikus.
- Súlyozott gráfnál a súlyt tároljuk.

Helyfoglalás: $|V|^2$ cella.

	1	2	3	4	5	6	7
1	0	1	0	1	0	0	0
2	0	0	1	0	1	0	0
3	0	0	0	0	0	1	0
4	0	1	0	0	1	0	0
5	0	0	0	0	0	1	1
6	0	0	0	0	0	0	1
7	0	0	0	0	0	0	0

Melyiket válasszuk?

Szempont	**lista**	**mátrix**
ritka gráf ($\|E\| \ll \|V\|^2$)	kedvező	pazarló
sűrű gráf	nagy mutatóteher	kedvező
`Szomszédok`	azonnali	$\|V\|$ lépés
`VezetÉl`	listában keresés	azonnali

Nagy gráfoknál a mátrix helyigénye miatt többnyire a lista marad.

Mire figyeljünk bejáráskor?

- A gráfban lehetnek **körök** – vigyázni kell, hogy ne kerüljünk végtelen ciklusba.
- A gráf nem feltétlenül **összefüggő** – tisztázni kell a célt: egy komponens bejárása vagy az összes csúcs elérése?
- Ezért minden bejárás nyilvántartja a **már elért** csúcsokat.

Mindkét alapbejárás egy adott `start` csúcsból indul, és az **onnan elérhető** csúcsokat dolgozza fel.

A szélességi bejárás elve

- Mindig a **legkorábban elért**, még át nem vizsgált csúcs szomszédai felé haladunk.
- S: sor (FIFO) – az elért, még fel nem dolgozott csúcsok.
- F: a már elért csúcsok halmaza.

Pszeudokód

```
eljárás Szélességi(start)
  S ⇐ start
  F := {start}
  ciklus amíg S ≠ ∅
    k ⇐ S
    Feldolgoz(k.tart)
    ciklus x ∈ k.Szomszédok
      ha x ∉ F akkor
        S ⇐ x
        F := F ∪ {x}
  ciklus vége
```

A csúcsok állapota a bejárás során

- **Elért, de fel nem dolgozott:** benne van az `S` sorban – a szomszédait még nem vizsgáltuk meg.
- **Elért és feldolgozott:** benne van `F`-ben, de már nincs `S`-ben – az összes kivezető élét feldolgoztuk.
- **Még el nem ért:** egyik adatszerkezetben sincs benne.

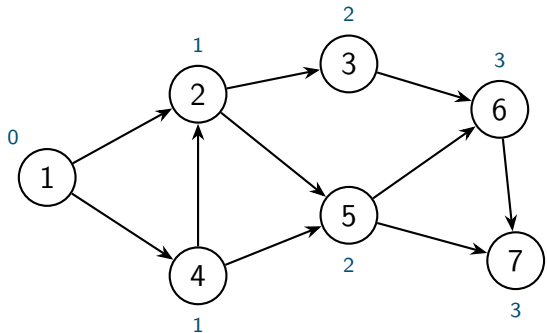
Kiegészítés: távolságok és feszítőfa

- `d[v]`: a `start`-tól való távolság,
  a **legkevesebb élből** álló út hossza
- `d[start] = 0`; új csúcsra `d[x] = d[k] + 1`
- $\pi[v]$: melyik csúcsból értük el v-t
- A π értékekből **szélességi feszítőfa** épül.

A ciklusmag kiegészítve

```
ciklus x ∈ k.Szomszédok
  ha x ∉ F akkor
    S ⇐ x
    F := F ∪ {x}
    d[x] := d[k] + 1
    π[x] := k
```

Példa: szélességi bejárás az 1. csúcsból



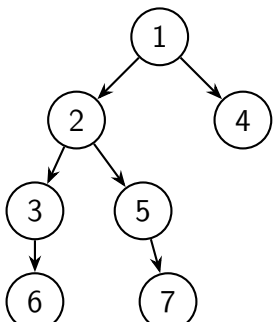
A csúcsok mellett a d érték (élek száma a legrövidebb úton).

A bejárás lépései

lépés	kivett `k`	új csúcsok	sor a lépés után
0	–	1	1
1	1	2, 4	2, 4
2	2	3, 5	4, 3, 5
3	4	–	3, 5
4	3	6	5, 6
5	5	7	6, 7
6	6	–	7
7	7	–	–

Feldolgozási sorrend: **1, 2, 4, 3, 5, 6, 7**

A szélességi fa



v	1	2	3	4	5	6	7
d	0	1	2	1	2	3	3
π	–	1	2	1	2	3	5

A π értékekből visszafelé lépkedve megkapjuk a legrövidebb (legkevesebb élű) utat.

A szélességi keresés alkalmazásai

- **Komponensek keresése:** a bejárás után `F` a `start`-ból elérhető csúcsok halmaza.
- **Erős összefüggőség:** a bejárást a **fordított** gráfon is lefuttatjuk;
  ha az is minden csúcsot elér, a komponens erősen összefüggő.
- **Útkeresés:** a π értékekből kiolvasható a legkevesebb élű út; egy kiegészítő leállási feltétellel a célcsúcs elérésekor megállhatunk.
- **Lépésszám:** minden csúcsot egyszer veszünk ki, minden élt egyszer nézünk meg $\Rightarrow$ $O(|V| + |E|)$.

A mélységi bejárás elve

- Mindig az **utoljára elért**, új kivezető éllel rendelkező csúcs felé haladunk.
- Ha nincs ilyen, **visszalépünk** egy korábbira.
- F: a már elért csúcsok halmaza.
- A visszalépést a **rekurzió verme** biztosítja.

Pszeudokód

```
eljárás MélyRek(k, címsz.  F)
  F := F ∪ {k}
  Feldolgoz(k.tart)
  ciklus x ∈ k.Szomszédok
    ha x ∉ F akkor
      MélyRek(x, F)
  ciklus vége
eljárás MélységiBejárás(start)
  F := ∅
  MélyRek(start, F)
```

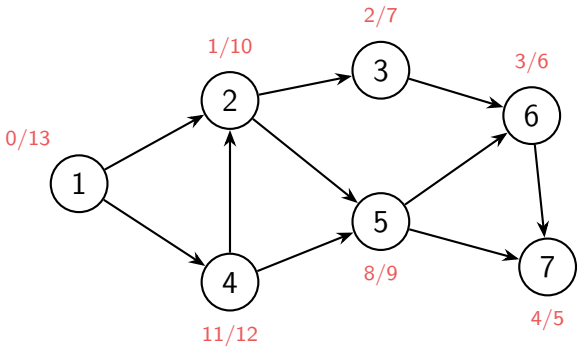
Be- és kilépési idők

- `t`: lépésszámláló (kezdetben 0)
- `be[k]`: mikor léptünk be k-ba
- `ki[k]`: mikor hagytuk el k-t
- A π tömb itt is felépíthető (mélységi feszítőfa).

Kiegészített változat

```
eljárás MélyRek(k, F, t, be,
ki)
  be[k] := t++
  F := F ∪ {k}
  ciklus x ∈ k.Szomszédok
    ha x ∉ F akkor
      π[x] := k
      MélyRek(x, F, t, be,
ki)
  ciklus vége
  ki[k] := t++
```

Példa: mélységi bejárás az 1. csúcsból



A csúcsok mellett `be`/`ki` idők.
Feldolgozási sorrend: **1, 2, 3, 6, 7, 5, 4**

A két bejárás összehasonlítása

	szélességi	**mélységi**
adatszerkezet	sor (FIFO)	verem (rekurzió)
haladási irány	legkorábban elért felé	utoljára elért felé
sorrend a példán	1, 2, 4, 3, 5, 6, 7	1, 2, 3, 6, 7, 5, 4
legrövidebb út élszámra	igen	nem
lépésszám	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$

Topologikus rendezés

Definíció

Egy irányított gráf **topologikus rendezése** a csúcsok olyan sorrendje,
amelyben minden (u, v) élre u megelőzi v-t.

- Csak akkor létezik, ha a gráf **nem tartalmaz irányított kört** (DAG).
- Klasszikus példa: öltözködés sorrendje, gyártási folyamat ütemezése.
- A példagráfunk egy topologikus rendezése: **1, 4, 2, 5, 3, 6, 7**

Topologikus rendezés mélységi bejárással

- A csúcsokat az **elhagyás fordított sorrendjében** soroljuk fel.
- Egyszerűbben: elhagyáskor a csúcsot láncolt lista **elejére** szúrjuk.
- Minden csúcsból el kell indítani a bejárást.

Pszeudokód

```
eljárás MélyRek(k, F, fej)
  F := F ∪ {k}
  ciklus x ∈ k.Szomszédok
    ha x ∉ F akkor
      MélyRek(x, F, fej)
  ciklus vége
  ListaElejére(fej, k)
függvény TopRendezés(G)
  ciklus st ∈ G.Csúcsok
    ha st ∉ F akkor
      MélyRek(st, F, fej)
  vissza fej
```

A mélységi keresés alkalmazásai

- **Visszalépéses keresés:** a lehetséges megoldások fájában haladunk előre,
  elakadás esetén visszalépünk.
- **Fabejárások:** a pre-, in- és post-order bejárás mind mélységi keresés.
- **Topologikus rendezés:** pl. gyártási folyamat ütemezése.
- **Erősen összefüggő komponensek:** bejárás kilépési idők megjegyzésével,
  majd a transzponált gráf bejárása fordított sorrendben.
- **Labirintus bejárása** – erről szól a következő fejezet.

A labirintusprobléma

- A labirintus azonosítható egy **összefüggő, véges, irányítatlan** gráffal:
  az elágazások és zsákutcák a csúcsok, a folyosók az élek.
- A különbség a korábbiakhoz képest: **nem ismerjük a térképet**.
  Mindig csak az aktuális csúcsról és éleiről tudunk valamit.
- A gráfot nem „kívülről” látjuk, hanem **bennük sétálunk** – az éleket krétával megjelölhetjük.
- Cél: a labirintus **összes** pontjának bejárása, majd visszatérés.

Az első megoldások: Wiener (1871), Tarry (1895), Trémaux (Kőnig Dénes könyve nyomán).

Wiener algoritmusa (1871)

- Haladunk új éleken, amíg zsákutcába vagy már látott csúcsba nem érünk.
- Ekkor **azonnal visszafordulunk**, és a megtett úton visszafelé lépkedünk,
  amíg olyan csúcsot nem találunk, amelynek van még használatlan éle.
- Onnan letérünk, és az egészet ismételjük.

Wiener tétele

A séta a kezdőcsúcsba vezet vissza, és a gráf **minden élét** bejárja.

Feltételezi, hogy a megtett útvonalat bármikor vissza tudjuk követni.

Tarry algoritmusa (1895) – a feltevések

Amikor egy P csúcsba érkezünk, tudnunk kell:

- mely P-hez csatlakozó éleken haladtunk már át, és **mely irányban**;
- melyik élen értük el **először** P-t – ez P **bejárati éle**.

Megjegyzés

A kezdőcsúcsnak nincs bejárati éle.

Tarry szabályai

1. szabály

Egyetlen élen sem haladunk át **kétszer ugyanabban az irányban**.

2. szabály

Egy csúcsot a **bejárati élén** csak akkor hagyunk el, ha a csúcs minden más élén már kiléptünk.

Amíg van más választás, mindig azt választjuk; a bejárati él a **végső tartalék**.

Tarry tétele

Tétel

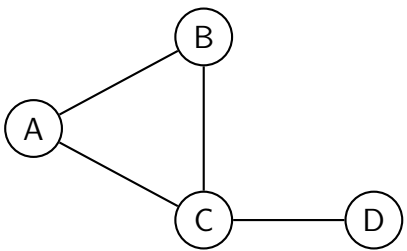
A Tarry-szabályokat követő séta **minden élen pontosan kétszer** halad át,
egyszer-egyszer a két irányban, és a **kezdőcsúcsban** ér véget.

- Így a séta hossza pontosan $2 \cdot |E|$ lépés.
- A bejárás a gráf **minden csúcsát** érinti (összefüggőség).
- Más csúcsnál mindig van hova továbbmenni: a be- és kilépések száma kiegyenlíti egymást.

Tarry: a bizonyítás gondolatmenete

- **Hol állhatunk meg?** Egy $V \neq A$ csúcsba az n-edik belépéskor V-t már $n-1$-szer elhagytuk, tehát van még szabad kijárata $\Rightarrow$ a séta **csak a kezdőcsúcsban** állhat le.
- **Minden élt bejárunk?** Jelöljük a csúcsokat felfedezési sorrendben: $A_0, A_1, \ldots$ Ha A_n-t a bejárati élén hagytuk el, előtte már minden más élén kiléptünk; indukcióval minden A_i minden élén átmentünk mindkét irányban.
- **Minden csúcsot elérünk?** Ha B-t nem, az összefüggőség miatt volna él bejárt és be nem járt csúcs között – ellentmondás.

Tarry: példa



Kezdőcsúcs: **A**

#	lépés	megjegyzés
1	$A \to B$	B bejárati éle: AB
2	$B \to C$	C bejárati éle: BC
3	$C \to A$	új él
4	$A \to C$	A-nak nincs bejárati éle
5	$C \to D$	új él
6	$D \to C$	D zsákutca
7	$C \to B$	csak a bejárati él maradt
8	$B \to A$	csak a bejárati él maradt

$|E| = 4$, a séta 8 lépés $= 2|E|$, és A-ban ér véget.

Trémaux algoritmusa – a feltevések

- Itt **nem** tételezzük fel, hogy ismerjük az áthaladás **irányát**.
- Csak annyit tudunk, hogy egy élen **hányszor** haladtunk át: 0, 1 vagy 2 jel van rajta.
- A jeleket az él **mindkét végén** látnunk kell.

A jelölés gyakorlata

Belépéskor és kilépéskor is jelet teszünk az élre, így az élek *jelöletlen* / *egyszer jelölt* / *kétszer jelölt* állapotúak lehetnek.

Trémaux szabályai

Tegyük fel, hogy az e élen érkeztünk a V csúcsba:

1. Ha V-ben **még nem jártunk**: folytassuk egy tetszőleges jelöletlen élen.
   Ha nincs ilyen (V zsákutca), forduljunk vissza e-n.
2. Ha V-ben **már jártunk**, és e-n most mentünk át **először**: **azonnal forduljunk vissza** e-n.
3. Egyébként: hagyjuk el V-t egy **jelöletlen** élen, ha van ilyen; ha nincs, akkor egy **egyszer jelölt** élen.
4. Ha ez sem lehetséges, a séta véget ér.

Trémaux tétele

Tétel

A séta a **kezdőcsúcsban** ér véget, és **minden élen pontosan kétszer** haladunk át – egyszer-egyszer a két irányban.

- A futási idő itt is pontosan $2 \cdot |E|$.
- **Kulcsállítás:** minden Trémaux-séta egyben **Tarry-séta** is, így Tarry tétele már garantálja a helyességet.
- A 2. szabály (azonnali visszafordulás) biztosítja, hogy egy csúcsot
  csak végső esetben hagyunk el a bejárati élén.

Trémaux algoritmusa F/E jelölésekkel

- F (first): azon él jele, amelyen **először** értük el a csúcsot
- E (exit): kilépésre használt él jele
- Jelet soha nem törlünk.

Pszeudokód

```
V := A
ciklus amíg V-nek van jelöletlen
   éle vagy F-fel jelölt éle:
   ha van jelöletlen e = (V,U):
      jelöljük e-t V-nél E-vel
      ha U-n nincs jelölés:
         jelöljük e-t U-nál F-fel
         V := U
      különben:  e U-nál E,
         (visszafordulunk)
   különben:  lépjünk az F élen
      V := U
```

A három labirintus-algoritmus

	Mit kell tudni?	Kulcsszabály
Wiener	vissza tudjuk követni az utat	azonnal visszafordulunk
Tarry	él iránya + bejárati él	bejárati él csak végső esetben
Trémaux	él jeleinek *száma*	új él, különben egyszer jelölt

- Mindhárom minden élen kétszer halad át: a lépésszám $\leq 2|E|$, Tarrynál és Trémaux-nál pontosan $2|E|$.
- **Trémaux algoritmusa a mélységi keresés (DFS) első változata** – a 19. századi labirintusprobléma és a mai gráfalgoritmus ugyanaz.

A feladat

Legrövidebb utak egy forrásból

Adott egy **élsúlyozott** G gráf, amelyben minden élsúly **nemnegatív**,
és egy s kezdőcsúcs. Keressük a legrövidebb utat s-ből **minden** csúcsba.

- A nemnegatív súlyok miatt nem kell negatív összsúlyú körrel számolnunk.
- Minden körben **egy újabb** csúcsra határozzuk meg a végleges távolságot –
  de előre nem tudjuk, milyen sorrendben.

A használt adatszerkezetek

- **KÉSZ:** azon csúcsok halmaza, amelyekre már tudjuk az s-től vett távolságot.
- **távolság[v]:** a végleges távolság, ha $v \in$ KÉSZ; egyébként ∞.
- **d[v]:** a KÉSZ-en kívüli v-re a legrövidebb olyan $s \to v$ út hossza,
  amely végig KÉSZ-ben halad, és csak az utolsó éllel lép ki:

$$d[v] = \min_{u \to v,\ u \in \text{KÉSZ}} \Big\{ \text{távolság}[u] + c(u, v) \Big\}$$

- **honnan[v]:** a v-t megelőző csúcs a legrövidebb ismert úton.

Miért helyes a d képlete?

- Egy ilyen út a KÉSZ-en belül eljut valamely u csúcsba – ennek a résznek a legrövidebb hossza `távolság[u]`.
- Innen a $c(u, v)$ súlyú uv éllel éri el v-t.
- A minimumot **minden** olyan KÉSZ-beli u-ra kell venni, ahonnan vezet él v-be,
  mert mindegyik lehetőséget figyelembe kell vennünk.

Az algoritmus menete

1. Kezdetben KÉSZ $= \{s\}$, `távolság[s] = 0`, minden másra ∞.
2. `d[s] = *`; `d[v] = c(s,v)`, ha van él s-ből v-be, különben ∞.
   `honnan[v] = s`, ha van ilyen él, különben $*$.
3. Amíg van KÉSZ-en kívüli csúcs véges d értékkel:
   - kiválasztjuk a **legkisebb** d értékű v^* csúcsot;
   - v^* KÉSZ-be kerül, `távolság[`v^*`] := d[`v^*`]`, `d[`v^*`] := *`;
   - minden KÉSZ-en kívüli w szomszédra: ha `távolság[`v^*`] + c(`v^*`,w) < d[w]`, akkor frissítünk.

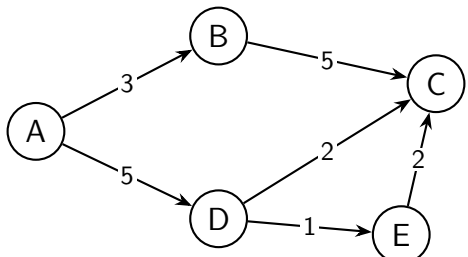
A pszeudokód: inicializálás

```
KÉSZ := {s}
távolság[v] := 0, ha v = s,
  egyébként távolság[v] := végtelen
d[v] := *, ha v = s;
  d[v] := c(s,v), ha van él s-ből v-be;
  különben d[v] := végtelen
honnan[v] := s, ha v = s vagy van él s-ből v-be;
  egyébként honnan[v] := *
```

A pszeudokód: a fő ciklus

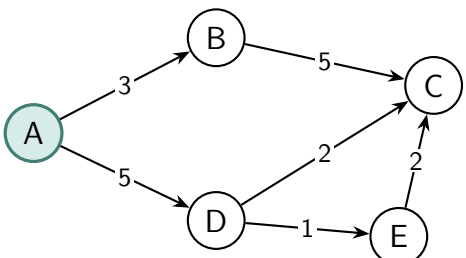
```
ciklus amíg van KÉSZ-en kívüli csúcs, ahol d[v] nem végtelen:
  v* := az a csúcs, ahol d[v] minimális
  v* KÉSZ-be megy
  távolság[v*] := d[v*];  d[v*] := *
  ciklus w = 1-től n-ig:
    ha A[v*,w] nem végtelen és d[w] nem *:
      ha távolság[v*] + c(v*,w) < d[w]:
        d[w] := távolság[v*] + c(v*,w)
        honnan[w] := v*
  ciklus vége
ciklus vége
```

A példagráf



Kezdőcsúcs: **A**. Minden élsúly nemnegatív.

Kezdeti állapot

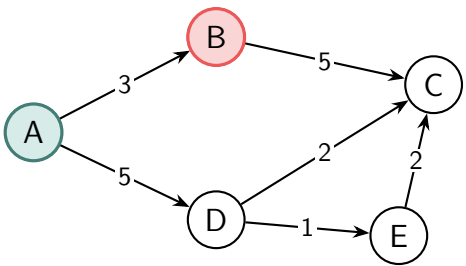


KÉSZ $= \{A\}$

	A	B	C	D	E
távolság	0	∞	∞	∞	∞
d	*	3	∞	5	∞
honnan	A	A	*	A	*

A kezdeti *d* értékek az *A*-ból kivezető élek súlyai.

1. lépés: B kerül KÉSZ-be

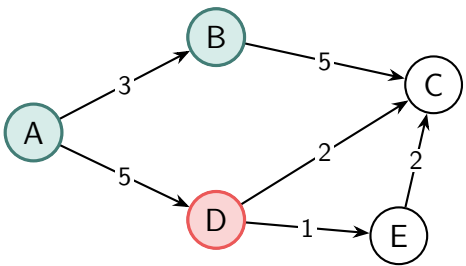


A legkisebb d érték B-nél van: `távolság[B] = 3`.

B KÉSZ-en kívüli szomszédja C: $3 + 5 = 8 < \infty \Rightarrow$ `d[C] = 8`, `honnan[C] = B`.

	A	B	C	D	E
távolság	0	3	∞	∞	∞
d	*	*	8	5	∞
honnan	A	A	B	A	*

2. lépés: D kerül KÉSZ-be

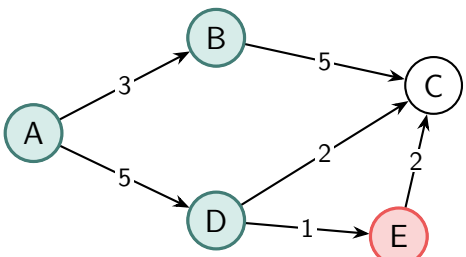


`távolság[D] = 5`. D szomszédai: C és E.

- $5 + 2 = 7 < 8 \Rightarrow$ `d[C] = 7`, `honnan[C] = D`
- $5 + 1 = 6 < \infty \Rightarrow$ `d[E] = 6`, `honnan[E] = D`

	A	B	C	D	E
távolság	0	3	∞	5	∞
d	*	*	7	*	6
honnan	A	A	D	A	D

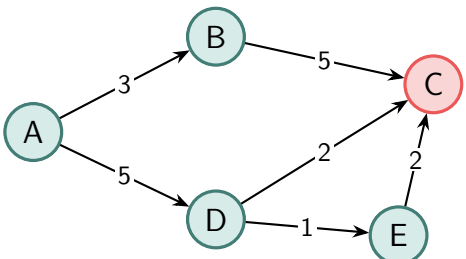
3. lépés: E kerül KÉSZ-be



`távolság[E] = 6`. E szomszédja C: $6 + 2 = 8$, de ez **nem** kisebb a jelenlegi $d[C] = 7$ értéknél
$\Rightarrow$ nincs módosítás.

	A	B	C	D	E
távolság	0	3	∞	5	6
d	*	*	7	*	*
honnan	A	A	D	A	D

4. lépés: C kerül KÉSZ-be – vége



`távolság[C] = 7`. *C*-nek nincs KÉSZ-en kívüli szomszédja.

	A	B	C	D	E
távolság	0	3	7	5	6
d	*	*	*	*	*
honnan	A	A	D	A	D

Nincs több véges *d* érték $\Rightarrow$ az algoritmus leáll.

Az eredmény

Legrövidebb utak az A csúcsból

csúcs	út	hossz
B	A–B	3
C	A–D–C	7
D	A–D	5
E	A–D–E	6

Az utakat a `honnan` értékekből kapjuk: a célcsúcsból visszafelé lépkedünk.
Pl. `honnan[C] = D`, `honnan[D] = A` $\Rightarrow$ az út A–D–C.

Helyesség: mit kell belátni?

Indukció a KÉSZ-be kerülés sorrendje szerint.

- **Alapeset:** `távolság[s] = 0` helyes, hiszen van 0 hosszú út s-ből s-be,
  és negatív él nincs, tehát rövidebb nem lehet.
- **Indukciós lépés:** ha a KÉSZ-beli csúcsokra helyes a távolság, akkor a következőnek beérkező v^*-ra is az. Ehhez két dolog kell:
  (A) van `távolság[`v^*`]` hosszú út s-ből v^*-ba;
  (B) minden $s \to v^*$ út legalább ilyen hosszú.

Helyesség: (A) van ilyen hosszú út

- A d definíciója szerint

$$d[v^*] = \text{távolság}[u^*] + c(u^*, v^*),$$

  ahol $u^* = \texttt{honnan}[v^*]$ a legjobb KÉSZ-beli megelőző csúcs.
- Az indukciós feltevés szerint u^*-ra a távolság helyes, tehát van $\texttt{távolság}[u^*]$ hosszú út s-ből u^*-ba.
- Ezt az utat az (u^*, v^*) éllel megtoldva kapunk egy $\texttt{távolság}[u^*] + c(u^*, v^*) = \texttt{távolság}[v^*]$ hosszú utat.

Helyesség: (B) rövidebb út nincs

Legyen U tetszőleges $s \to v^*$ út abban a pillanatban, amikor v^*-ot beválasztjuk.

- U a KÉSZ-ben indul és a KÉSZ-en kívül végződik, tehát van egy **első** éle, amely kilép KÉSZ-ből: legyen ez (u', v').
- Az $s \to u'$ szakasz hossza legalább $\texttt{távolság}[u']$, a $v' \to v^*$ szakaszé pedig nemnegatív (nincs negatív él).
- Így $|U| \geq \texttt{távolság}[u'] + c(u', v') \geq d[v'] \geq d[v^*]$, hiszen v^*-ot épp a legkisebb d értéke miatt választottuk.
- Tehát $|U| \geq \texttt{távolság}[v^*]$. ■

Lépésszám

- A kezdeti feltöltés: három n méretű tömb $\Rightarrow O(n)$.
- A fő ciklus magja legfeljebb n-szer fut le (minden körben egy csúcs kerül KÉSZ-be).
- A ciklusmagban: minimumkeresés a d tömbben $O(n)$, majd a szomszédok frissítése $O(n)$.

Összesen

$$n \cdot \Big(O(n) + O(1) + O(n)\Big) = O(n^2)$$

Kupaccal (prioritási sorral) megvalósítva $O\Big((|V| + |E|) \log |V|\Big)$ is elérhető.

Dijkstra és a szélességi bejárás

	szélességi bejárás	**Dijkstra**
élsúlyok	nincsenek (mind 1)	nemnegatív súlyok
következő csúcs	a sor eleje	a legkisebb d értékű
adatszerkezet	sor	prioritási sor / tömb
eredmény	legkevesebb élű út	legkisebb összsúlyú út

Ha minden élsúly 1, a két algoritmus ugyanazt adja:
a szélességi bejárás a Dijkstra-algoritmus speciális esete.

Összefoglalás

- **Tárolás:** szomszédsági lista (ritka gráf) vagy csúcsmátrix (sűrű gráf).
- **Szélességi bejárás:** sor, élszám szerinti legrövidebb utak, $O(|V| + |E|)$.
- **Mélységi bejárás:** verem/rekurzió, be- és kilépési idők, topologikus rendezés.
- **Tarry és Trémaux:** bejárás a gráf ismerete nélkül, minden élen pontosan kétszer, $2|E|$ lépés – Trémaux algoritmusa a DFS őse.
- **Dijkstra:** nemnegatív élsúlyok, körönként egy csúcs véglegesítése, $O(n^2)$.

Gyakorló feladatok

1. Futtassa le a szélességi és a mélységi bejárást a példagráfon a 4. csúcsból!
2. Adja meg a példagráf összes topologikus rendezését!
3. Járja be Tarry algoritmusával a teljes négyszögű gráfot (K_4)! Hány lépés a séta?
4. Mutassa meg példával, hogy Dijkstra algoritmusa negatív élsúly esetén hibás eredményt adhat!
5. Hogyan módosítaná a Dijkstra-algoritmust, hogy a legrövidebb utak *számát* is megadja?

Források és ajánlott irodalom

- Csima Judit: *Dijkstra algoritmus*. BME SZIT, 2019.
- Szénási Sándor: *Gráfok 1. – Tárolási módok, bejárások*. Óbudai Egyetem, NIK. `nik.uni-obuda.hu/sztf2`
- Dénes Kőnig: *Theory of Finite and Infinite Graphs*. Birkhäuser, 1990.
- G. Tarry: Le problème des labyrinthes. *Nouvelles Annales de Mathématiques*, 14, 1895.
- E. W. Dijkstra: A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 1959.